

# Taller de Observabilidad

OpenTelemetry + LGTM Stack + Práctica en  
Aula

 60 minutos

Cátedra de Ingeniería y Calidad de Software —  
2026

---

center

## 🕒 0-20' — Teoría

0-2' ¿Qué es observabilidad?

2-5' Monitoreo vs 3 pilares

5-10' 📈 Métricas + 📝 Logs

10-13' 🔍 Trazas

13-17' OTel + LGTM + Pipeline

17-20' Arquitectura del taller

```
<style> .timeline { display: flex; flex-direction: column; gap: 6px; } .tl-item { display: flex; align-items: center; gap: 10px; padding: 6px 12px; border-radius: 6px; background: var(--gray-50); font-size: 0.85em; } </style>
```

🎯 **Objetivo:** Aplicar observabilidad a tu TP con Grafana + OpenTelemetry

## 🕒 20-60' — Laboratorio

| Min   | Paso                     | Slides |
|-------|--------------------------|--------|
| 20-25 | Setup + Grafana          | 23-26  |
| 25-33 | Métricas + Dashboard RED | 27-31  |
| 33-37 | Logs con Loki            | 32-34  |
| 37-40 | Trazas con Tempo         | 35-37  |
| 40-50 | Ejercicios               | 38-41  |
| 50-55 | Aplicación al TP         | 42     |
| 55-60 | Cierre                   | 43-44  |



**Grot dice:** Hoy vamos a ver cómo conectar la teoría de calidad con la práctica real. ¡Preparen sus consolas!

## 0-20' — Teoría

Conceptos fundamentales de observabilidad

# ¿Qué es Observabilidad?

## Definición

El término proviene de la **teoría de control**: un sistema es observable si podés determinar su **estado interno** midiendo sus **salidas externas**.

“**Observabilidad** es la capacidad de entender el estado interno de un sistema a partir de sus señales externas (telemetría).”

**Pregunta clave:** Si tu app se cae en producción, ¿cuánto tardás en saber por qué?

## ¿Por qué ahora?

Los sistemas modernos son:

| Ayer                     | Hoy                   |
|--------------------------|-----------------------|
| Monolítico               | Microservicios        |
| Servidores fijos         | Containers dinámicos  |
| Despliegues semanales    | Deploys múltiples/día |
| Equipo pequeño           | Equipos distribuidos  |
| "Funciona en mi máquina" | "Funciona en K8s"     |

Sin observabilidad → **caja negra**

## ¿Por qué importa?

### Sin observabilidad...

- Error en producción → revisar logs manualmente
- Usuario reporta lentitud → no sabés por dónde empezar
- Microservicio falla → no sabés cuál
- Pico de tráfico → no sabés si es normal

**MTTR (tiempo de reparación):** horas o días

### Con observabilidad...

- Error en producción → traza con causa raíz en segundos
- Lentitud → span lento identificado al instante
- Microservicio falla → dashboard con alerta contextual
- Pico de tráfico → métricas RED en tiempo real

**MTTR (tiempo de reparación):** minutos

 **SWEBOK v4:** La observabilidad se integra en **Software Engineering Operations** (KA 06): telemetría pervasiva en todas las capas del stack (aplicación, SO, servidor) con ciclos de retroalimentación continua.

# Monitoreo vs Observabilidad

## ● Monitoreo

"Saber si algo anda mal"

- Definís **qué** medir de antemano
- Dashboards fijos
- Preguntas conocidas: "¿está caído?"
- Alertas predefinidas
- **Reactivo**
- Te dice el **síntoma**

“

*"El servidor respondió 503"*

”

## ● Observabilidad

"Entender por qué anda mal"

- Hacés **preguntas nuevas** sin deployar
- Exploración interactiva
- Preguntas: "¿por qué está lento?"
- Descubrís lo inesperado
- **Proactivo**
- Te dice la **causa raíz**

“

*"503 porque la DB está saturada"*

”

💡 El monitoreo te dice **QUE** algo anda mal. La observabilidad te dice **POR QUÉ**.



**Grot dice:** El monitoreo te dice *qué* falló. La observabilidad te permite investigar *por qué*.

## Los 3 Pilares



### Métricas

Datos cuantitativos

- Counters, Gauges
- Histograms
- Framework **RED**
- Automáticas con OTel



### Logs

Eventos con timestamp

- JSON estructurado
- Con `trace_id`
- Filtrables por `event`
- Niveles: info, warn, error



### Trazas

Recorrido de peticiones

- Spans con duración
- Árbol de llamadas
- Atributos de negocio
- Tracing distribuido

 **La correlación entre las 3 señales se logra mediante el `trace_id`** — mismo ID en métricas, logs y trazas.

## Métricas: Tipos

| Tipo             | Ejemplo             | Comportamiento  |
|------------------|---------------------|-----------------|
| <b>Counter</b>   | http_requests_total | Solo incrementa |
| <b>Gauge</b>     | memory_usage_bytes  | Sube y baja     |
| <b>Histogram</b> | http_duration_ms    | Distribución    |

```
# Rate: requests/segundo
rate(http_server_duration_ms_count[5m])

# Latencia percentil 95
histogram_quantile(0.95,
  rate(http_duration_bucket[5m]))
```

## Frameworks de Métricas

### RED — PARA SERVICIOS

| Señal           | Consulta                        |
|-----------------|---------------------------------|
| <b>Rate</b>     | rate(metric[5m])                |
| <b>Errors</b>   | rate(metric{status=~"5.."}[5m]) |
| <b>Duration</b> | histogram_quantile(0.95, ...)   |

### USE — PARA INFRAESTRUCTURA

| Señal              | Pregunta              |
|--------------------|-----------------------|
| <b>Utilization</b> | ¿Qué % de CPU/RAM?    |
| <b>Saturation</b>  | ¿Hay colas de espera? |
| <b>Errors</b>      | ¿Errores de sistema?  |



## 4 Señales Doradas (SRE)

### Las 4 que importan

Google SRE define que **todo servicio** debería medir:

| Señal             | Pregunta                     |
|-------------------|------------------------------|
| <b>Latencia</b>   | ¿Cuánto tarda en responder?  |
| <b>Tráfico</b>    | ¿Cuántas peticiones/segundo? |
| <b>Errores</b>    | ¿Cuántos requests fallan?    |
| <b>Saturación</b> | ¿Qué tan cerca del límite?   |

“If you measure 100 metrics, you measure none.  
Pick the 4 that matter.”

### En nuestro taller:

| Señal    | Query Mimir                                     |
|----------|-------------------------------------------------|
| Rate     | <code>rate(http...count[5m])</code>             |
| Errors   | <code>rate(http...{status=~\"4..\"}[5m])</code> |
| Duration | <code>histogram_quantile(0.95, ...)</code>      |

 RED en acción



**Grot dice:** Si tenés que elegir qué medir hoy en tu TP, empezá por la Latencia y la Tasa de Errores. ¡Es el core del RED!



## Logs: Texto vs Estructurados

### ✗ Texto libre

```
2024-05-29 Post creado  
2024-05-29 Error: validation failed
```

- ✗ No se puede filtrar por campo
- ✗ Sin correlación con trazas
- ✗ Difícil de analizar

### ✓ JSON estructurado

```
{  
  "event": "post.created",  
  "postId": "01HXYZ...",  
  "trace_id": "abc123...",  
  "span_id": "def456..."  
}
```

- ✓ Filtrable por event, postId
- ✓ Correlacionado con trazas
- ✓ Analizable automáticamente



## El Logger Pino con `mixin`

Configuramos Pino para inyectar automáticamente el contexto de OpenTelemetry en cada log:

```
// observability/logger.ts
import pino from "pino"
import { trace, context } from "@opentelemetry/api"

export const logger = pino({
  transport: { target: "pino-pretty" },
  level: "info",
  mixin() {
    // 1. Obtener span activo en el contexto
    const span = trace.getSpan(context.active())
    if (!span) return {}

    // 2. Extraer IDs de traza y span
    const ctx = span.spanContext()
    return {
      trace_id: ctx.traceId,
      span_id: ctx.spanId
    }
  }
})
```

## ¿Por qué funciona?

- **Mixin dinámico**: se ejecuta en tiempo de logeo, no al iniciar la app.
- **Context propagation**: captura el span exacto que se está ejecutando en ese hilo.
- **Correlación automática**: sin que el developer tenga que pasar el `trace_id` como argumento manualmente.

```
logger.info("Operación exitosa")
// Genera:
// { "msg": "Operación exitosa", "trace_id": "abc...", "span_id": "def..." }
```



## Buenas Prácticas de Logs

Cada log debe tener:

### Un event único

user.created  
payment.failed  
order.shipped  
auth.login

Buscable y agrupable

### El trace\_id siempre

```
{  
  "event": "post.created",  
  "trace_id": "abc..."  
}
```

1 click → de log a traza

### Niveles consistentes

| Nivel | Significado       |
|-------|-------------------|
| info  | Operación normal  |
| warn  | Algo inesperado   |
| error | Afecta al usuario |

**Regla de oro:** Si un log no tiene `event` y `trace_id`, no es un log estructurado. Es ruido.



**Grot dice:** Un log sin `trace_id` es como un ticket sin número: no podés seguirle el rastro en el sistema.

## Trazas: Anatomía

### Una traza = árbol de spans

```
GET /api/posts - 8ms      ← TRACE
├── [HTTP] router          ← SPAN (2ms)
│   method=GET, route=/api/posts
├── [App] listar-posts     ← SPAN (5ms)
│   posts.count=3
│   ├── [DB] SELECT *      ← SPAN (1ms)
│   │   db.system=postgres
│   └── [HTTP] response    ← SPAN (1ms)
│       status_code=200
```

### Componentes de un Span

| Componente        | Descripción                    |
|-------------------|--------------------------------|
| <b>Trace ID</b>   | UUID único de toda la petición |
| <b>Span ID</b>    | ID de cada operación           |
| <b>Parent ID</b>  | ID del span padre (jerarquía)  |
| <b>Name</b>       | crear-post , listar-posts      |
| <b>Duration</b>   | Tiempo en milisegundos         |
| <b>Attributes</b> | post.id , http.method          |
| <b>Status</b>     | OK / ERROR                     |
| <b>Events</b>     | Excepciones, logs internos     |

# Trazabilidad Distribuida

Taller de Observabilidad con Grafana

## El trace\_id viaja con la petición



### ¿Por qué es importante?

- Seguir una petición **entre servicios**
- Identificar **cuellos de botella**
- Reducir **MTTR** (tiempo de reparación)
- Entender **causa raíz** de fallos

### ¿Dónde se propaga?

Via headers HTTP estándar:

```
traceparent: 00-abc123...-def456...-01
```

- **00**: versión
- **abc123...**: trace\_id
- **def456...**: span\_id
- **01**: trace flags

# OpenTelemetry: El Estándar

center

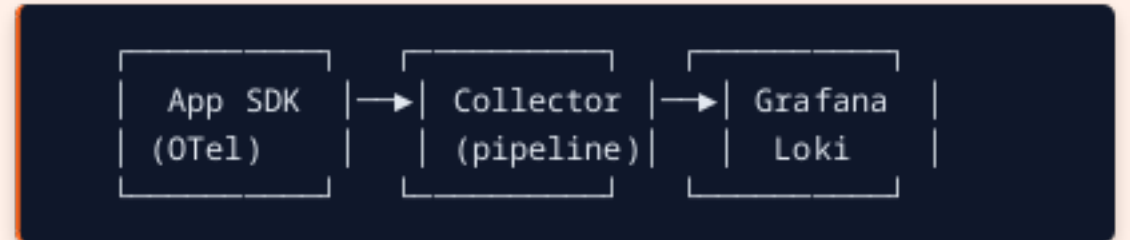
## Una sola especificación

**Antes de OTel:** cada proveedor exigía su propio SDK propietario (`dd-trace`, `newrelic`, etc.), generando un fuerte **vendor lock-in**.

**Con OTel:** instrumentás tu código una sola vez usando un estándar abierto y podés exportar los datos a cualquier destino compatible.

## La solución de la industria

- **Agnóstico:** neutral frente a proveedores
- **Unificado:** un solo SDK para trazas, métricas y logs
- **Comunidad:** el segundo proyecto más activo de la CNCF (luego de Kubernetes)



**Grot dice:** OpenTelemetry es el estándar de facto de la industria. Lo usás hoy y mañana podés cambiar de Grafana a Datadog sin tocar tu código.

## OpenTelemetry: API + SDK + Collector



### API

Interfaz estándar

Tracer

Meter

Logger

Solo definiciones  
Independiente del proveedor



### SDK

Implementación

Pipeline de datos

Exporters OTLP

Auto-instrumentaciones



### Collector

Servidor aparte

Recibe OTLP

Procesa (filtra, agrega)

Exporta a 1+ backends

 **OTLP** (OpenTelemetry Protocol) es el protocolo estándar para transportar telemetría. Soporta gRPC (4317) y HTTP (4318).



# Auto-instrumentación: Sin Código

Taller de Observabilidad con Grafana

## El SDK de OpenTelemetry

Configuración en `telemetry.ts` que orquesta los exporters y las auto-instrumentaciones:

```
// observability/telemetry.ts
import { NodeSDK } from "@opentelemetry/sdk-node"
import { getNodeAutoInstrumentations } from
"@opentelemetry/auto-instrumentations-node"
import { OTLPTraceExporter } from
"@opentelemetry/exporter-trace-otlp-grpc"

export function initTelemetry(): void {
  const sdk = new NodeSDK({
    resource: new Resource({
      [SEMRESATTRS_SERVICE_NAME]: "taller-backend",
    }),
    traceExporter: new OTLPTraceExporter({
      url: "http://otel-collector:4318/v1/traces"
    }),
    instrumentations: [getNodeAutoInstrumentations()],
  })
  sdk.start()
}
```

## En acción:

Sin OTel:  
POST /api/posts → ??? → ??? → ??? → 201

Con OTel (automático):  
POST /api/posts → [HTTP: 1ms] →  
[Fastify: 1ms] → [PG: 3ms] → 201

💡 Esto funciona porque OTel hace **monkey-patching** de las librerías estándar en runtime al iniciar. No modifica tus archivos fuente.

# Instrumentación Manual: Datos de Negocio

Taller de Observabilidad con Grafana

## El Wrapper `startSpan`

Encapsulamos la complejidad de crear, cerrar y capturar excepciones en un utilitario reutilizable:

```
// observability/tracer.ts
import { trace, Span, SpanStatusCode } from "@opentelemetry/api"

export function startSpan<T>({
  name: string,
  fn: (span: Span) => Promise<T>
}): Promise<T> {
  const tracer = trace.getTracer("backend")
  return tracer.startActiveSpan(name, async (span) => {
    try {
      const result = await fn(span)
      span.setStatus({ code: SpanStatusCode.OK })
      return result
    } catch (err) {
      span.setStatus({
        code: SpanStatusCode.ERROR,
        message: err.message
      })
      throw err
    } finally {
      span.end()
    }
  })
}
```

## Uso en las Rutas

Envolvemos las llamadas del controlador para enriquecer las trazas automáticas con metadata del dominio:

```
// http/fastify/routes/posts.ts
app.post("/api/posts", async (request, reply) => {
  const input = request.body
  return startSpan("crear-post", async (span) => {
    span.setAttribute("post.title", input.title)
    span.setAttribute("post.author", input.author)

    const post = await createPost.execute(input)
    span.setAttribute("post.id", post.id)

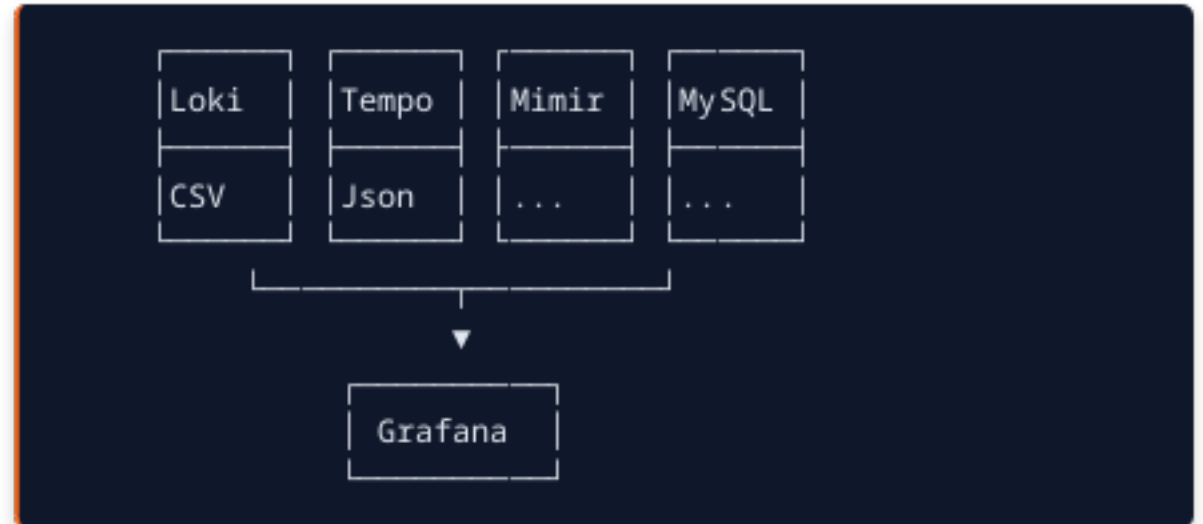
    logger.info({
      event: "post.created",
      postId: post.id
    }, "Post creado")
    return reply.status(201).send(post)
  })
})
```

## Stack LGTM

| Componente | Señal         | Puerto | Función                    |
|------------|---------------|--------|----------------------------|
| Loki       | Logs          | 3100   | Almacena logs con labels   |
| Grafana    | Visualización | 3000   | Dashboards + Explore       |
| Tempo      | Trazas        | 3200   | Rastreo distribuido        |
| Mimir      | Métricas      | 9009   | TSDB Prometheus-compatible |

### Filosofía "Big Tent"

Grafana visualiza datos **donde residen**:



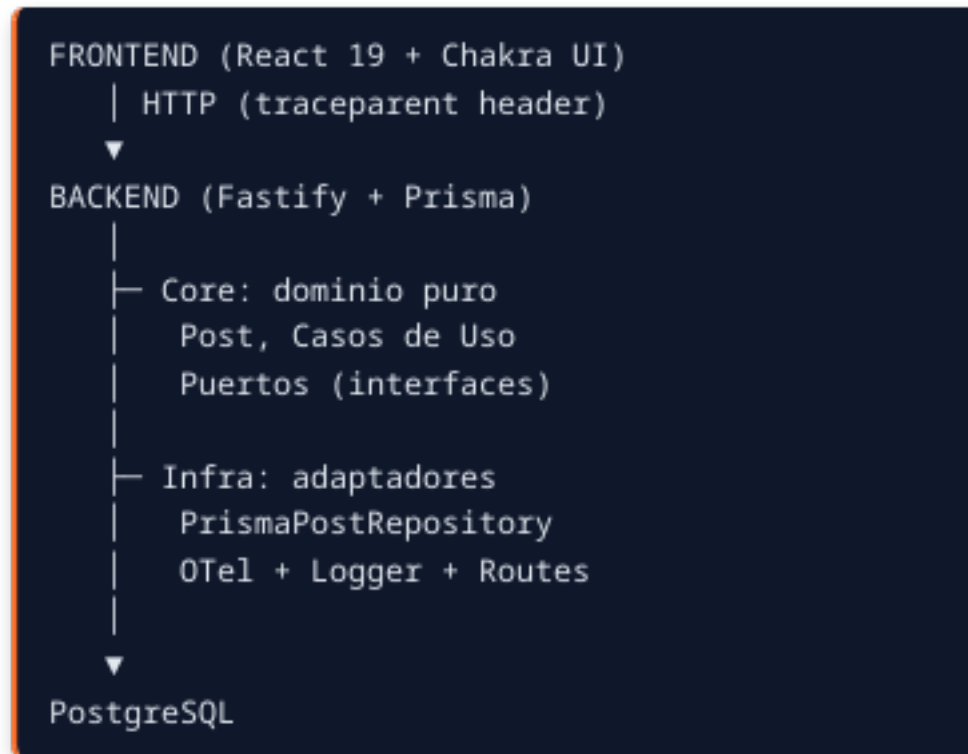
✓ **Provisioning:** Datasources y dashboards se configuran desde archivos YAML/JSON montados como volúmenes. **Sin clicks manuales.**

# Pipeline Completo

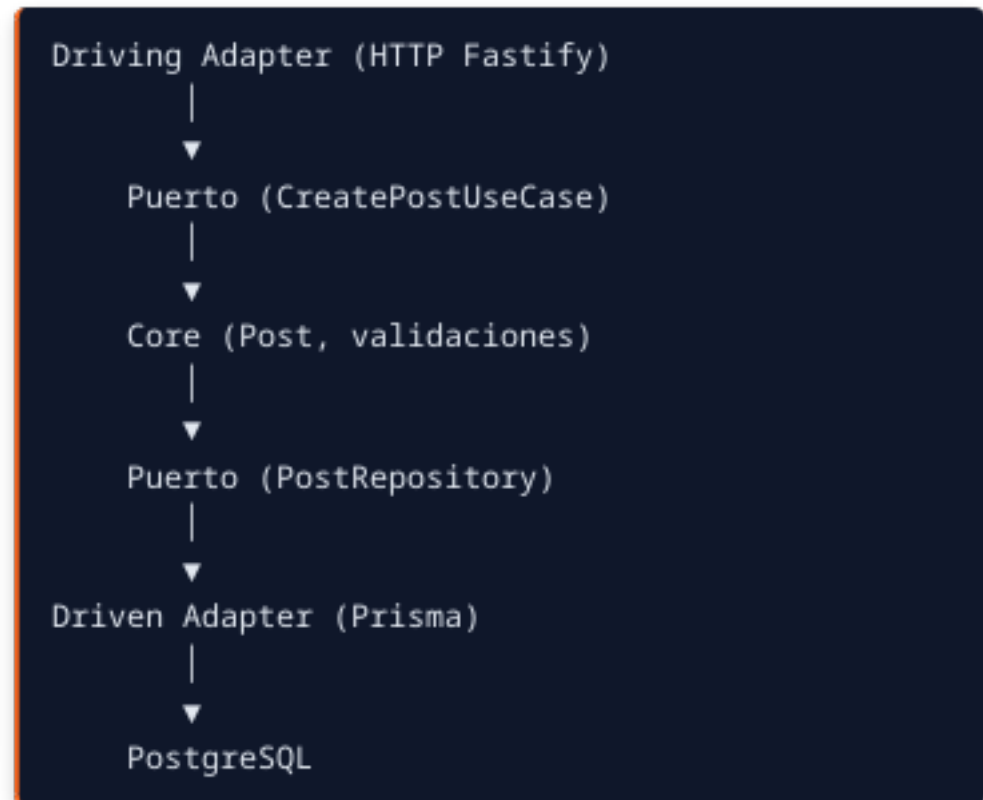


# Nuestra Arquitectura

## Stack del Taller



## Principio Hexagonal



Cada capa se instrumenta por separado

## Lo que Genera la App Automáticamente



### Métricas

Cada request:

- `http...count +1`
- `http..._bucket`

Cada 5s → Mimir

RED automático ✨



### Logs

Cada `logger.info()` :

```
{
  "event": "post.created",
  "trace_id": "abc..."
}
```

Tiempo real → Loki



### Trazas

Cada request:

```
POST /api/posts
├ [HTTP] router
├ [App] crear-post
├   └ [DB] prisma
└ [HTTP] response
```

Inmediato → Tempo

📌 **El alumno NO escribe código de observabilidad.** La app ya genera todo: trazas, métricas y logs via OTel. Su tarea es explorar, consultar queries y crear dashboards.



## 20-55' — Laboratorio

Práctica guiada — Stack funcionando con 1 comando

## Qué necesitás

- 1 **Docker 24+** con Docker Compose v2
- 2 **Git**
- 3 **Navegador** (Chrome / Firefox)
- 4 **~4-6 GB RAM** libre para el stack

## Comandos

```
# 1. Clonar
git clone https://github.com/rodriguezemaun/
taller-observabilidad-grafana.git

cd taller-observabilidad-grafana/docker

# 2. ¡Un solo comando!
docker compose up -d

# 3. Verificar
docker compose ps
```

## 7 servicios que se levantan

| Servicio                        | Puerto    | Para qué                                                                                         |
|---------------------------------|-----------|--------------------------------------------------------------------------------------------------|
| <b>App</b> (Frontend + Backend) | 3001      | La app del taller                                                                                |
| <b>PostgreSQL</b>               | 5432      | Base de datos                                                                                    |
| <b>OTel Collector</b>           | 4317-4318 | Pipeline de telemetría                                                                           |
| <b>Loki</b>                     | 3100      |  Logs       |
| <b>Tempo</b>                    | 3200      |  Trazas     |
| <b>Mimir</b>                    | 9009      |  Métricas   |
| <b>Grafana</b>                  | 3000      |  Dashboards |





## 20-23': Paso 1 — Explorar la App

### 1.1 Abrir la app

```
http://localhost:3001
```

Vas a ver:

- **Lista de posts** (vacía al inicio)
- **Pestaña "Crear Post"**
- UI con Chakra UI

### 1.2 Creá 3-4 posts

Con distintos títulos y autores.

### 1.3 Verificar via API

```
curl http://localhost:3001/api/posts
```

```
[
  {
    "id": "01KSR...",
    "title": "Mi primer post",
    "author": "Alumno",
    "createdAt": "2026-05-29..."
  }
]
```



La app ya genera métricas, logs y trazas via OTel. Sin que hayas escrito código de observabilidad.



**Grot dice:** Asegúrense de crear al menos 3 posts con distintos autores para ver cómo Mimir y Loki segmentan los datos.



## 23-25': Paso 2 — Grafana

### 2.1 Abrir Grafana

```
http://localhost:3000
```

Usuario: `admin` / Contraseña: `admin`

### 2.2 Ver datasources

Menú → Connections → Data Sources

- ✓ **Loki** (logs)
- ✓ **Tempo** (trazas)
- ✓ **Mimir** (métricas)

Ya configurados sin clicks.

### ¿Cómo funciona provisioning?

```
# docker/grafana/datasources.yml
apiVersion: 1
datasources:
  - name: Loki
    type: loki
    url: http://loki:3100
  - name: Tempo
    type: tempo
    url: http://tempo:3200
  - name: Mimir
    type: prometheus
    url: http://mimir:9009/prometheus
```

📁 Los dashboards también se auto-cargan desde archivos JSON en `docker/grafana/dashboards/` — 3 dashboards listos para explorar.

## 25-28': Paso 3 — Métricas con Mimir

### 3.1 Explore → Mimir

```
# RATE – requests/segundo  
rate(http_server_duration_ms_count[5m])
```

Una línea que sube y baja con el tráfico.

### 3.2 Generar tráfico

Creá posts mientras mirás la gráfica. La línea sube.

### 3.3 Errors + Duration

```
# ERRORS – tasa de error 4xx  
rate(http_server_duration_ms_count{  
  http_status_code=~"4.." }[5m])
```



**Probar:** Creá un post SIN título → error 400 → aparece en Errors

```
# DURATION – latencia p95  
histogram_quantile(0.95,  
  rate(http_server_duration_ms_bucket[5m]))
```

## 28-30': Consultas con Filtros

### Filtrar por endpoint

```
# Rate solo para POST
rate(http_server_duration_ms_count{
  http_request_method="POST"
}) [5m])

# Rate solo para GET
rate(http_server_duration_ms_count{
  http_request_method="GET"
}) [5m])
```

### Filtrar por status

```
# Solo status 200 (éxito)
rate(http_server_duration_ms_count{
  http_status_code="200"
}) [5m])

# Solo status 400 (error)
rate(http_server_duration_ms_count{
  http_status_code="400"
}) [5m])
```

💡 Los filtros `{}` en PromQL son clave para hacer preguntas específicas. Sin filtros: todas las métricas juntas. Con filtros: métricas segmentadas por endpoint, método, status, etc.

## 🕒 30-33': Paso 4 — Dashboard RED

### 4.1 Crear dashboard

1. Menú → Dashboards → **New Dashboard**
2. **+ Add visualization**
3. Datasource: **Mimir**
4. Query: `rate(http...count[5m])`
5. Panel title: "Rate de Requests"
6. **Apply**

### 4.2 Agregar 2 paneles más

| Panel           | Query                                      |
|-----------------|--------------------------------------------|
| Tasa de Errores | <code>rate(...{status=~"4.."}[5m])</code>  |
| Latencia p95    | <code>histogram_quantile(0.95, ...)</code> |

### RED Dashboard



● **RED** = Rate, Errors, Duration — framework SRE

## 33-35': Paso 5 — Logs con Loki

### 5.1 Explore → Loki

```
# Todos los logs del backend  
{service_name="taller-backend"}
```

Todos los logs aparecen en vivo.

### 5.2 Ver en tiempo real

Creá un post desde la UI. El log aparece instantáneamente en Loki.

### 5.3 Filtrar por evento

```
# Solo creaciones exitosas  
{service_name="taller-backend"  
  |= "post.created"  
  
# Solo errores  
{service_name="taller-backend"  
  |= "error"  
  
# Error de validación  
{service_name="taller-backend"  
  |= "post.validation_error"  
  
# Listado de posts  
{service_name="taller-backend"  
  |= "post.listed"
```

## 35-37': Filtrar Logs por Trace ID

### 5.4 Correlación logs ↔ trazas

Cada log contiene `trace_id`. Usalo para vincular:

```
# Buscar logs de una traza específica
{service_name="taller-backend"}
|= "01HXYZ..." - trace_id
```

### ¿Cómo obtener el trace\_id?

1. Andá a Tempo (Explore)
2. Abrí cualquier traza
3. Copiá el `trace_id`
4. Pegalo en Loki

### Eventos en nuestra app

| Evento                             | Cuándo ocurre       |
|------------------------------------|---------------------|
| <code>post.created</code>          | Creación exitosa    |
| <code>post.listed</code>           | Listado de posts    |
| <code>post.found</code>            | Búsqueda por ID     |
| <code>post.validation_error</code> | Error de validación |
| <code>post.not_found</code>        | Post inexistente    |
| <code>post.error</code>            | Error interno       |

```
{
  "event": "post.created",
  "postId": "01KSR...",
  "trace_id": "abc...",
  "level": 30
}
```

## 🕒 37-39': Paso 6 — Trazas con Tempo

### 6.1 Explore → Tempo

```
{service.name="taller-backend"}
```

### 6.2 Explorar una traza

Click en cualquier traza. Ves la estructura completa:

```
POST /api/posts
├─ [HTTP] request      ← 2ms
├─ [Fastify] handler   ← 1ms
├─ [App] crear-post    ← 5ms ← manual
│   ├── post.title="..."
│   ├── post.author="..."
│   └─ [DB] prisma      ← 1ms
└─ [HTTP] response     ← 1ms
```

### 6.3 Consultas útiles

```
# Spans lentos (>100ms)
{service.name="taller-backend"
  | span.duration > 100ms

# Trazas con error
{service.name="taller-backend"
  | span.status = error

# Spans de negocio específicos
{service.name="taller-backend"
  && name="crear-post"
```

💡 Los atributos de negocio ( `post.title` , `post.author` ) se agregan con `span.setAttribute()` en los spans manuales.



## 🕒 39-42': Paso 7 — Dashboard Completo

### 7.1 Agregar panel de logs

1. Dashboard → **+ Add**
2. Datasource: **Loki**
3. Query: `{service_name="taller-backend"}`
4. Panel type: **Logs**
5. **Apply**

### 7.2 Agregar panel de trazas

1. Dashboard → **+ Add**
2. Datasource: **Tempo**
3. Query: `{service.name="taller-backend"}`
4. Panel type: **Trace List**
5. **Apply**

### Dashboard Final



 **3 señales en 1 dashboard:** métricas  
RED + logs + trazas

## 42-46': Ejercicios 1 y 2

### Ejercicio 1: Correlación

Vincular logs con trazas

1. En Tempo, abrí una traza
2. Copiá el `trace_id`
3. En Loki, buscá: `{...} |= "<trace_id>"`
4. **Respondé:** ¿Qué información adicional tiene el log que no está en la traza?

 Correlación logs ↔ trazas en acción

 2 minutos

### Ejercicio 2: Error Forzado

Ver un error en las 3 señales

1. Creá un post **sin título** desde la UI
2. **Mimir:** ejecutá la query de errores 4xx
3. **Loki:** buscá `|= "validation_error"`
4. **Tempo:** `| span.status = error`
5. **Respondé:** ¿Qué señal te da más información para debuggear?

 3 minutos



**Grot dice:** La correlación logs-trazas es magia pura. ¡Hagan click en el `trace_id` en Loki y vean cómo se abre Tempo al lado!

## 46-50': Ejercicios 3 y 4

### Ejercicio 3: Cuello de Botella

Identificar lentitud en el sistema

1. Buscá en Tempo: `| span.duration > 50ms`
2. ¿Hay spans lentos?
3. ¿Cuál es el span más lento?
4. **Respondé:** ¿Cómo mejorarías ese tiempo?

 Pensá como un SRE

 2 minutos

### Ejercicio 4: Dashboard Propio

Crear un dashboard desde cero

1. Creá un **nuevo** dashboard
2. Agregá un panel de métrica a elección
3. Agregá un panel de logs filtrado por `post.created`
4. Agregá un panel de trazas lentas
5. **Guardalo como "Mi Dashboard"**

 4 minutos — el más completo



## 50-55': Aplicar a tu TP

### Lo que necesitás

- 1 **Docker Compose** en tu proyecto
- 2 **Node.js** / Python / Java
- 3 **3-6 horas** de trabajo
- 4 Agregar **OTel SDK**
- 5 Reemplazar **console.log** por logger
- 6 Agregar servicios **LGTM** al compose

### Pasos concretos

```
# 1. Copiar servicios LGTM del taller
#   a tu docker-compose.yml

# 2. Instalar OTel
npm install @opentelemetry/api \
  @opentelemetry/sdk-node \
  @opentelemetry/auto-instrumentations-node

# 3. Crear telemetry.ts + logger.ts
#   (copiar de packages/infrastructure/src/observability/)

# 4. Inicializar ANTES del servidor
initTelemetry()

# 5. console.log → logger.info()
#   con event + trace_id

# 6. docker compose up
```



**Guía completa:** [notas-academicas/13-agregar-observabilidad.md](#) — Checklist + tiempos por stack

## Resumen: Conceptos Clave



### Métricas

Framework RED  
Rate, Errors,  
Duration  
Automáticas con OTel



### Logs

JSON estructurados  
Con `event` único  
Con `trace_id`



### Trazas

Spans con duración  
Atributos de negocio  
Tracing distribuido



### Correlación

`trace_id` en TODAS  
las señales  
→ Dashboard  
unificado

### ◆ OpenTelemetry

Estándar neutral  
1 instrumentación  
Cualquier backend



### Docker

7 servicios LGTM  
1 comando:  
`docker compose up`



### Grafana

Provisioning  
automático  
Sin clicks manuales  
Dashboards pre-  
cargados



### Hexagonal

Capas separadas  
Cada capa  
instrumentada  
Dominio puro sin  
OTel

## Recursos

### Notas del Taller

notas-academicas/ :

| Archivo                      | Tema                                                                                                      |
|------------------------------|-----------------------------------------------------------------------------------------------------------|
| 12-integracion-conceptos.md  | Mapa conceptual completo                                                                                  |
| 11-otel-instrumentacion.md   | OTel paso a paso                                                                                          |
| 06-grafana-docker.md         | Grafana + Docker                                                                                          |
| 13-agregar-observabilidad.md |  Guía para aplicar al TP |
| 05-arquitectura-hexagonal.md | Puertos y adaptadores                                                                                     |

### Links útiles

| Recurso      | URL                                                                                                                                   |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Repositorio  | <a href="https://github.com/rodriguezemaun/taller-observabilidad-grafana">github.com/rodriguezemaun/taller-observabilidad-grafana</a> |
| OTel Docs    | <a href="https://opentelemetry.io/docs/">opentelemetry.io/docs/</a>                                                                   |
| Grafana Docs | <a href="https://grafana.com/docs/">grafana.com/docs/</a>                                                                             |
| Grafana Play | <a href="https://play.grafana.org">play.grafana.org</a>                                                                               |

### Libros

- **SRE Book** — Beyer et al. (O'Reilly)
- **The Art of Monitoring** — Turnbull
- **Clean Architecture** — Martin
- **SWEBOK v4** — IEEE Computer Society

## 55-60': Conclusiones

### Qué aprendimos hoy

| Concepto                                                                                              | Lo aplicamos en...    |
|-------------------------------------------------------------------------------------------------------|-----------------------|
|  Métricas RED        | Mimir + PromQL        |
|  Logs estructurados  | Loki + LogQL          |
|  Trazas distribuidas | Tempo + TraceQL       |
|  Correlación         | trace_id en 3 señales |
|  Dashboards          | RED + Logs + Trazas   |
|  Docker              | LGTM en 7 servicios   |

### Para seguir explorando

1. Agregá observabilidad a tu **TP integrador** (guía en nota 13)
2. Explorá **Grafana Play** ([play.grafana.org](https://play.grafana.org))
3. Leé el **SRE Book** de Google
4. Probá **Pyroscope** (perfilado continuo)
5. Investigá **Grafana Alloy** (reemplazo de Promtail)

*"The best time to add observability was yesterday. The second best time is now."*

# Gracias

**La observabilidad no es un producto.**

**Es una capacidad de ingeniería.**

“*You can't improve what you can't measure.*” — Peter Drucker”

“*A monolith that is well-observed is better than 50 microservices that are not.*” — Charity Majors”

center

**¿Preguntas?**

Ingeniería y Calidad de Software — 2026